Ahmedabad University

Subject - Digital Image Processing

8. Vehicle Orientation Estimation Using Conventional Image Processing

Weekly Report - 4

Date of Submission: 17/10/2026

Repo Link: https://github.com/encrypted-official/DIP_2026_Project8_Group1

Group - 1

Enrollment No.	Name of the Student	Name of the Program
AU2440150	Vehan Gajjar	Btech CSE
AU2340012	Vedant Solanki	Btech CSE
AU2440173	Abhi Jain	Btech CSE
AU2440242	Krish Prajapati	Btech CSE

2026 (Monsoon Semester)

1. Work done this week

After the mid-sem presentation we had five weeks left including this one (Week 4 to Week 8). The next plan we showed in the mid-sem had quite a lot in it, so instead of starting everything together we broke it into parts and gave each week one clear target. Table 1 shows how we split it. For this week the target was to go back to the mid-sem results properly, write Method C, and get the ablation ready so that next week is only about running things and reading the numbers.

Table 1: Plan for the remaining five weeks

Week	Part of the plan	What we want at the end
4	Re-check mid-sem results, write Method C, set up ablation	Method C code with tests, ablation script
5	Run Method C and the ablation on the 40 image set, then the full val split	Updated results table, best settings for each method
6	Front vs back using intensity along the axis	First full 0 to 360 heading results
7	Tuning on train_original, error analysis by size, shadow and occlusion	Error breakdown tables and failure figure
8	Final evaluation on the full val split	Final report and presentation

Looking back at the mid-sem results: For the mid-sem we ran PCA and Hough on the first 40 images of the validation split. After removing difficulty 1 vehicles this gave 1822 crops. The numbers are repeated in Table 2 so we can compare against them in the coming weeks.

Table 2: Mid-sem results on 40 validation images (1822 crops)

Metric	PCA	Hough
Coverage (%)	99.89	99.84
MAE (°)	11.15	14.10
Median error (°)	6.28	6.82
P90 error (°)	27.21	39.13
Within 10° (%)	65.22	62.67

When we looked at the table again with a bit of time, a few things stood out. Both methods gave an answer for nearly every crop, only 2 crops for PCA and 3 for Hough were left without one.

The median error of the two is almost the same (6.3 and 6.8 degrees), so for a normal crop both do a similar job. The real gap is in the bad cases. The P90 of Hough is about 12 degrees worse than PCA, and that tail is what pulls its MAE up. Also, around one in three crops is still more than 10 degrees off for both methods. So the problem is not the average car, it is a group of crops where the axis goes badly wrong, mostly the small ones under 20 px (MAE above 30 degrees) and the ones with shadows or road lines.

A problem in our angle convention: While sorting the overlay images into good and bad cases, we saw that in some of the failures the green and red lines were not random. They looked like mirror images of each other about the horizontal, like the SUV in Figure 1 where the red line follows the body but the green one does not. We traced this back to our angle folding. The rule $\varphi = \min(r, 180 - r)$ sends both 30 and 150 degrees to 30, but these are two different axes, one goes down to the right and the other goes up to the right. Our estimators give an angle from 0 to 180, so a correct prediction of 150 degrees on a vehicle whose true axis is 150 got compared with 30 and was counted as 60 degrees off. The overlay script also drew the folded angle, which is why the green line looked flipped.



We fixed this by comparing axes in the 0 to 180 range, using heading mod 180 as the truth. Since heading_deg is already saved in metadata.csv, evaluate.py now builds the truth from it directly and we did not need to cut the crops again. We added a unit test for the mirror case as well. This means the mid-sem numbers in Table 2 have to be run again before we compare any new method with them. We have not done the re-run yet, so we are not putting any new numbers in this report. We also still need to check by hand whether the dataset measures heading clockwise or anticlockwise, because the y-axis of an image points down. If it is the other way round the truth becomes (- theta) mod 180.

Method C, Otsu mask with moments and minimum-area rectangle: Method C works on regions instead of edges. The idea came from the P90 failures, where the edges of the roof and the windscreen were stronger than the edges along the body and pulled the axis sideways. If we use the whole vehicle blob, those inner edges should matter much less. The steps in estimate_moments.py are: the same blur and CLAHE as before, Otsu thresholding, and then a choice of which side of the threshold is the vehicle. A car can be brighter or darker than the road, so we keep the side that covers the centre of the crop and not its border. After that a 3x3 opening and closing cleans the mask, and we keep only the connected part that is closest to the crop

centre. If this part is less than 10% of the crop area we return no prediction, because it is usually a windscreen or a shadow patch and not the car.

From this mask we read the axis in two ways. The first uses the second order central moments. The second fits the minimum-area rectangle around the blob and takes the direction of its long side. For the rectangle we work out the angle from the four corner points ourselves, because the angle that OpenCV returns for minAreaRect changed its convention in newer versions. If the long side is less than 1.15 times the short side the blob is almost square and we give no answer. Before trying it on real crops we tested it on synthetic images, a rotated rectangle on a flat grey background at 7 angles between 0 and 160 degrees, both bright car on dark road and dark car on bright road. Both ways stay within 3 degrees in all these cases, and a square blob or a flat crop correctly gives no prediction. This only tells us the code and the angle handling are right; real crops will be much harder.

Ablation setup: Last time every setting was written directly inside the code. Now preprocess.py takes the blur size, CLAHE on or off and the two Canny thresholds as inputs. The default values are the same as in the mid-sem, so the old results do not change. The new ablation.py runs every method under Canny (30, 90), (50, 150) and (80, 200), CLAHE on and off, and blur sizes 1, 3 and 5. That is 18 settings each for PCA and Hough. Method C does not use Canny, so it gets 6, which makes 48 runs in total. For each run it saves coverage, MAE, median, P90 and within 10 degrees, along with the MAE for crops under 20 px, 20 to 40 px and above 40 px. Each crop is read from disk only once, so the full grid on 1822 crops should not take too long.

Small fix: One of us had OpenCV 5 installed and the Hough script crashed for them, because HoughLinesP now returns the lines in a different array shape. We changed it to work with both versions. All 21 unit tests pass now.

2. Work to be done next week

Next week is the second half of the Method C and ablation part of the plan. We plan to:

- Check the heading direction of the dataset by hand on around 20 crops, and fix the sign if needed.
- Run PCA and Hough again on the same 40 images (1822 crops) with the corrected angle, so that the old and new numbers can be compared fairly.
- Run Method C with both the moments and the rectangle reading on the same crops and add them to the results table.
- Run the full ablation grid and pick the best setting for each method, mainly looking at P90 and within 10 degrees and not only MAE.

- If time allows, move from 40 images to the full validation split.
- Start putting the bad crops into groups such as tiny vehicle, shadow, road marking and touching neighbour, so that the error analysis in Week 7 is quicker.

3. Literature Review

This week we read mainly about thresholding and shape fitting, since Method C depends on getting a clean mask of the vehicle.

- Otsu (1979) picks the threshold that gives the largest variance between the two classes of the grey level histogram. It needs no setting from us, which is why we chose it, but it assumes the histogram has two clear groups. A grey car on a grey road does not give that.
- Sezgin and Sankur (2004) compared around 40 thresholding methods on different kinds of images. Otsu was not the best everywhere, and local methods did better when lighting is uneven. We kept this in mind as a possible change if shadows keep breaking our masks.
- Freeman and Shapira (1975) proved that the smallest rectangle around a closed shape has one side on the convex hull of that shape. Toussaint (1983) later used this in the rotating calipers method, which is the reason a minimum-area rectangle can be found quickly. This is what OpenCV's minAreaRect does on our contour.
- Suzuki and Abe (1985) gave the border following algorithm behind OpenCV's findContours. Reading it helped us understand why we only take the outer contours, as holes like the windscreen would otherwise come back as separate shapes.
- Gonzalez and Woods (2018), chapter on morphological processing, explain opening and closing. Opening removes small bright noise and closing fills small gaps. We use a small 3x3 kernel, because a bigger one would join two parked cars that are close to each other.

4. Challenges Faced

- Finding the mirror problem in our own angle convention after the mid-sem was a bit disappointing, because it means the results we presented have to be computed again. We still think it was better to catch it now than in the final week.
- Deciding which side of the Otsu threshold is the car. White and black cars are easy, but silver and grey cars are close to the road colour and the mask often comes out broken or empty for them.
- Shadows. For dark cars the shadow falls on the same side of the threshold as the car, so the blob becomes wider in the direction of the sun and the axis can turn towards the shadow.

- Cars parked very close together can join into one blob after thresholding. Choosing the part closest to the centre helps, but when they actually touch it does not.
- Different OpenCV versions on our laptops. The minAreaRect angle and the HoughLinesP output both behave differently between versions, and this cost us some time before we found it.
- Splitting the remaining weeks. Front vs back heading is the part we are least sure about, so we gave it its own week and kept the final week only for evaluation and writing.

References

[1] N. Otsu, "A Threshold Selection Method from Gray-Level Histograms," IEEE Transactions on Systems, Man, and Cybernetics, vol. 9, no. 1, pp. 62-66, 1979.

[2] M. Sezgin and B. Sankur, "Survey over Image Thresholding Techniques and Quantitative Performance Evaluation," Journal of Electronic Imaging, vol. 13, no. 1, pp. 146-165, 2004.

[3] H. Freeman and R. Shapira, "Determining the Minimum-Area Encasing Rectangle for an Arbitrary Closed Curve," Communications of the ACM, vol. 18, no. 7, pp. 409-413, 1975.

[4] G. T. Toussaint, "Solving Geometric Problems with the Rotating Calipers," in Proc. IEEE MELECON, Athens, 1983.

[5] S. Suzuki and K. Abe, "Topological Structural Analysis of Digitized Binary Images by Border Following," Computer Vision, Graphics, and Image Processing, vol. 30, no. 1, pp. 32-46, 1985.

[6] R. C. Gonzalez and R. E. Woods, Digital Image Processing, 4th ed., Pearson, 2018.

[7] M.-K. Hu, "Visual Pattern Recognition by Moment Invariants," IRE Transactions on Information Theory, vol. 8, no. 2, pp. 179-187, 1962.

[8] DRASHTI-HaOBB dataset, Zenodo, https://zenodo.org/records/18278989